GAMECA: an accessible, HPC-integrated pipeline for the downstream analysis of transposable-element families

Anmol Dash

July 4, 2026

Abstract

Transposable elements (TEs) make up a large fraction of mammalian genomes and are increasingly recognised as functional contributors to genome regulation and evolution [1]. Mature tools exist for the *discovery* of TE families from raw assemblies [2], but the *downstream* characterisation of a known family—grouping its copies by sequence, relating them to transcription-factor binding and gene ontology, profiling expression, and designing family-specific primers—still requires assembling many independent tools and ad-hoc scripts. We present **GAMECA** (Gene Alignment, Motif, Expression and Clustering Analysis), a modular pipeline that chains these downstream steps behind a single interface and runs them either locally or on LSF/Slurm high-performance-computing (HPC) clusters. A native desktop application drives a Python backend over a streaming protocol, and the same engine is exposed as a Nextflow DSL2 workflow [3] that follows nf-core conventions [4], so the copy-resolved analysis modules run as parallel, individually-resourced, resumable tasks and the whole pipeline can be embedded as a subworkflow in other DSL2 pipelines. Engineering for the practical bottlenecks—an in-memory genome cache that removes repeated multi-gigabyte FASTA reads, plus optional compiled hot paths—keeps whole-family analysis tractable. We position GAMECA against both TE-specific tools and general analysis platforms, showing that it uniquely packages the downstream TE-family workflow behind an accessible, HPC-ready interface, and illustrate it end-to-end on the *MT2_Mm* mouse ERV family using locus-level expression quantified by SQuIRE [5]. GAMECA is open source.

1 Introduction

Interspersed repeats derived from TEs account for roughly half of the human genome [6] and a comparable or larger share of many other eukaryotic genomes [1], where they shape genome organisation, supply regulatory sequence, and influence host gene expression [7]. The natural unit of analysis is the *family*: it groups copies descended from a common ancestral element, and most biological questions about activity, regulation and lineage are posed at the family level.

The bioinformatics of TEs has historically concentrated on two upstream problems—building libraries of family models *de novo* from an assembly and annotating a genome against them—and tools such as RepeatModeler2 [2] and the RepeatMasker/Dfam ecosystem [8] address them well. What remains comparatively under-served is the *downstream* workflow that begins once a family has been named: extracting its loci and sequences, partitioning them into sub-groups, aligning them and building consensus models, testing for enriched transcription-factor binding sites and associated gene functions, profiling expression across conditions, and designing locus- or family-specific primers. In practice these steps are stitched together from separate command-line tools

with incompatible conventions, and the resulting pipelines are hard to reproduce and hard to move between a laptop and a cluster.

GAMECA applies the philosophy of integrated analysis platforms [3, 4, 9, 10]—hide infrastructure, make every step reproducible, run anywhere—to the specific, recurring problem of downstream TE-family analysis. It is not a TE-discovery tool and does not compete with RepeatModeler2 [2]; it takes a family name and an assembly as its starting point and automates the characterisation that follows, with first-class support for the LSF and Slurm schedulers that dominate academic HPC.

2 How GAMECA differs from existing tools

GAMECA's closest neighbours fall into two groups that, between them, leave the downstream characterisation of a named TE family largely unserved (Table 1).

TE-specific tools cover the upstream or a single stage. RepeatModeler2 performs *de novo* discovery of TE families and emits consensus models [2], which the RepeatMasker/Dfam ecosystem then uses to annotate a genome [8]. Both produce inputs to—rather than instances of—family characterisation; GAMECA in fact consumes the RepeatMasker/Dfam locus coordinates as its starting point. SQuIRE addresses a single downstream stage, locus-specific repeat expression [5], and GAMECA consumes its counts rather than competing with it. None of these tools clusters a family into sub-groups, builds per-group consensus, tests motif/GO enrichment and designs family-specific primers behind one interface.

General platforms supply infrastructure, not a TE workflow. Galaxy is a browser-accessible workbench for reproducible analysis [9]; GenePattern packages analyses as reusable modules [10]; and nf-core/Nextflow makes container-backed pipelines portable across most HPC schedulers and cloud providers [3, 4]. These are powerful substrates, but none ships a ready-made downstream-TE-family workflow: the user must still locate, wrap and wire each stage and supply the TE-specific logic. GAMECA does not compete with this substrate—it builds on it: the pipeline ships a Nextflow DSL2 implementation (Section 6) that follows the nf-core meta and module conventions and is importable as a subworkflow, so GAMECA supplies exactly the TE-specific workflow that the general platforms leave to the user while inheriting their portability and reproducibility.

GAMECA's niche is therefore the otherwise-empty intersection of downstream-TE-family coverage and accessible, HPC-ready execution. The comparison is one of *scope and access*, established from each tool's documented capabilities; runtime measurements in Section 5 concern GAMECA's own engineering and are not cross-tool benchmarks.

3 Design and architecture

GAMECA is organised in three layers (Figure 1). A native desktop application built with Tauri and a React/Tailwind front end gives non-programmers point-and-click access to every pipeline step. The desktop shell does not perform analysis; it launches a bundled Python "sidecar" process and communicates with it over newline-delimited JSON on standard streams. Each user action becomes a request; the sidecar replies with a stream of log, progress and result events, so long-running jobs report live status and can be cancelled. This separation keeps the scientific code in plain Python—usable on its own from the command line—while the UI remains a thin, replaceable client.

The analysis backend consists of a driver script (`query.py`) that orchestrates the core stages,

Tool	TE-family focus	Downstream character.	Graphical interface	Built-in LSF/Slurm	Local + cluster, 1 UI	Reprod. environ.
GAMECA (this work)	✓	✓	✓	✓	✓	✓
RepeatModeler2 [2]	✓	×	×	×	×	○
RepeatMasker/Dfam [8]	✓	×	×	×	×	○
SQuIRE [5]	✓	○	×	×	×	○
Galaxy [9]	×	○	✓	○	○	✓
GenePattern [10]	×	○	✓	○	○	✓
nf-core/Nextflow [4]	×	○	×	✓	✓	✓

Table 1: Capability comparison on documented features. ✓ built-in; ○ partial, or possible but the user must assemble/configure it; × not provided. *TE-family focus*: purpose-built for TE families (RepeatModeler2 for discovery, RepeatMasker/Dfam for annotation, SQuIRE for expression). *Downstream characterisation*: the named-family workflow of clustering, consensus, motif/GO, expression and primers behind one interface. No cross-tool runtime is claimed.

one single-responsibility module per stage, a family of standalone `run_*.py` analysis modules, and an SSH client that connects the workflow to a remote cluster. In *local mode* GAMECA executes entirely on the user's machine, fetching sequence from the UCSC API [11] when no local FASTA is available. In *HPC mode* it uploads the analysis code to a login node, submits a batch job, streams the job's output back, and retrieves the results. Each external resource—RMSK/Dfam loci, UCSC sequence, JASPAR motifs [12], mygene.info annotation [13]—is reached through a thin adapter, so the rest of the pipeline is source-agnostic and adding an assembly or swapping a source is a localised change.

Orchestration is deliberately decoupled from the analysis code. The driver can run the downstream modules itself as a sequential subprocess loop, or delegate them to a Nextflow DSL2 layer [3] that executes the same unmodified Python modules as parallel, individually-scheduled tasks (Section 6). No scientific code is duplicated between the two: the Python modules remain the single implementation, and the Nextflow layer is a thin wrapper that calls them.

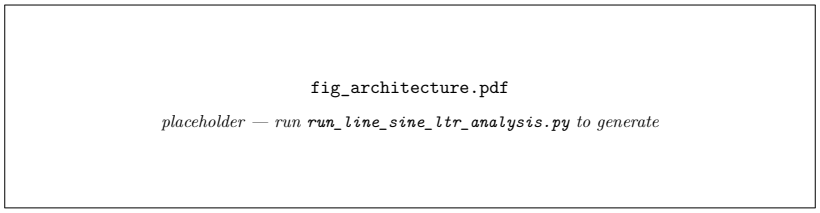

Figure 1: GAMECA's three-layer architecture. A Tauri/React desktop shell drives a Python analysis sidecar over a streaming NDJSON protocol; the backend runs locally or stages and submits jobs to an LSF/Slurm cluster. Coordinate, sequence, motif and annotation data are pulled from RMSK/Dfam, UCSC, JASPAR and mygene.info through uniform adapters.

4 The analysis pipeline

The pipeline is a directed sequence of stages (Table 2); each stage reads the tabular output of the previous one and writes a checkpoint, so a run can be resumed or a single stage re-executed without repeating the rest.

Stage	Function	Key methods / tools
Prep	Loci + sequences + divergence	RMSK / Dfam REST [8]; FASTA or UCSC [11]; Kimura milliDiv
Clustering	Sub-group the copies	k-mers → SVD → UMAP [14] → HDBSCAN [15]; adaptive min_*cluster_s*
Alignment	Consensus per group	MAFFT [16] + CIAlign [17]
Motif	Enriched TFBS	JASPAR [12] ∩ loci (BEDTools [18]) + Fisher
GO	Gene/ontology context	mygene.info [13]
Expression	Per-group profiles + DE	locus counts (e.g. SQuIRE [5]); Wilcoxon + BH-FDR between clusters
Primers	Family-specific primers	k-mer candidates + genome-wide specificity

Table 2: The GAMECA pipeline stages. Each stage consumes a tabular checkpoint and emits one, allowing partial re-runs.

Most stages are thin orchestration over established tools, with two exceptions worth detailing. *Clustering* is alignment-free: each sequence is encoded as a k-mer count vector (default k=6, a 4^6=4096-dimensional sparse vector built with scikit-learn [19]), the sparse matrix is reduced to 50 components by truncated SVD, embedded in two dimensions with UMAP [14], and finally partitioned with HDBSCAN [15]. The key HDBSCAN parameter min_*cluster_size* is set automatically: starting at $\lfloor N/5 \rfloor$, the pipeline decrements the divisor ($N/6$, $N/7$, …, $N/15$) and also sweeps UMAP `n_neighbors` $\in \{15, 30\}$, stopping at the first configuration that yields $\geq$2 clusters. The accepted divisor and neighbour count are recorded in `measured_values.tex` for reproducibility. This avoids the cost of a family-wide multiple alignment and scales to tens of thousands of loci (very large families are embedded from a fitted subset). *Repeat landscape*: the Prep stage retains the CpG-corrected Kimura divergence (milliDiv) reported by RepeatMasker/Dfam for each locus; the pipeline plots the divergence histogram per family, giving the evolutionary-age distribution of loci that is standard in TE methods papers. *Differential expression*: the Expression stage supplements per-cluster boxplots with a formal Wilcoxon rank-sum test between every cluster pair for each stage column, with Benjamini–Hochberg FDR correction; significant comparisons (q<0.05) are highlighted in the heatmap output. *Primer design* proposes family primers from frequent k-mers and then checks each candidate's specificity against the whole genome, scoring off-target hits across all chromosomes; this genome-wide search is the most I/O-intensive part of the pipeline and motivates the caching described next. The remaining stages—preparation, alignment/consensus, JASPAR motif enrichment with per-cluster Fisher exact tests, gene ontology lookup, and expression profiling from user-supplied per-locus counts—use the tools listed in Table 2 with the conventional defaults documented in the GAMECA source.

5 HPC integration and performance

Scheduler-agnostic execution. The HPC client connects to a login node over SSH (supporting key, password and keyboard-interactive/2-factor authentication), detects whether the cluster runs LSF (`bsub`) or Slurm (`sbatch`), stages the analysis code into a writable work directory, and submits a job built for the detected scheduler. The lifecycle is uniform across schedulers—authenticate, stage, build the environment, submit, stream progress, retrieve outputs—and the client runs either in an interactive streaming mode, relaying output line-by-line, or in a batch mode that submits and later retrieves results.

An in-memory genome cache. The dominant cost in whole-family primer design is reading the reference genome. A naive implementation re-reads the multi-gigabyte FASTA for every candidate

primer search; for a family with many candidates this means the genome is read from disk many times over. GAMECA instead parses the reference once into an in-memory `GenomeCache`—reused for both sequence extraction and all primer searches—and persists the parsed structure as a pickle so that subsequent runs skip parsing entirely. Figure 2 (left) reports the effect measured on the cluster: collapsing (pending run) repeated reads into a single in-memory load yields a $(pendingrun)\times$ reduction in genome-access time.

Compiled hot paths. A small number of batch operations—GC-content, length tabulation, and k-mer matrix construction—dominate the per-sequence arithmetic. GAMECA provides Cython [20] implementations of these (with the generated C source committed so nodes need only a C compiler, not Cython itself), and the pipeline falls back to pure Python when the extension is absent. Figure 2 (right) shows the measured speedups on real family sequences: $(pendingrun)\times$ for batch GC-content and $(pendingrun)\times$ for k-mer matrix construction.

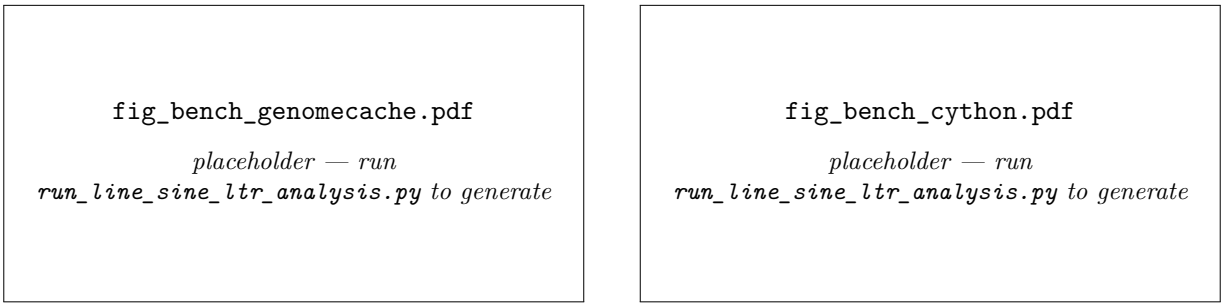

Figure 2: Performance engineering, measured live on the cluster. **Left**: genome-access time when the reference is re-read per search versus loaded once into `GenomeCache`. **Right**: runtime of the Cython hot paths versus their pure-Python equivalents on real family sequences. Ratios are written from the run.

Scaling with input size. To characterise how wall time, peak memory, and disk footprint grow with family size, the full pipeline (UCSC sequence fetch through Stage 11 and the LaTeX report) was run from scratch on MT2_Mm (mm10) at five `--max-loci` caps, each a fresh LSF job (job 97784432, 4 cores, 24 GB requested, single-threaded UCSC/JASPAR fetch as the bottleneck rather than CPU-parallel stages). Table 3 reports the numbers written by `query.py` to `stage_times.json` (wall time via `ru_maxrss`) together with the two dominant on-disk artefacts logged at the motif stage: the per-run JASPAR bigBed subset and the TE×motif overlap table.

Loci cap	Sequences	Wall time	Peak RSS	JASPAR BED	Overlap table
100	100	9.8 min	0.62 GB	7.6 MB	11.4 MB
500	500	40.0 min	1.33 GB	37.8 MB	57.5 MB
1000	1000	70.0 min	2.20 GB	75.2 MB	114.4 MB
2000	2000	134.3 min	3.91 GB	150.7 MB	228.8 MB
all	2641	175.4 min	4.96 GB	197.7 MB	299.8 MB

Table 3: Wall time, peak resident memory, and the two largest per-run on-disk artefacts as a function of `--max-loci`, MT2_Mm on mm10. The motif/JASPAR stage dominates wall time at every size (e.g. 8190 s of the 10522 s total at $N = 2641$), driven by the per-run bigBed query against JASPAR (no `tabix`/`bgzip` available on this node, so the intersect fell back to a full-file `bedtools` scan); peak memory and the overlap-table size both scale roughly linearly with N.

Two follow-on fixes are implied directly by this run. First, `tabix`/`bgzip` were absent on the compute node, so JASPAR intersection fell back to a full-file `bedtools` scan at every size instead of the seek-based `tabix` subsetting the motif stage otherwise prefers (Section 4); provisioning those binaries (or the container image that bundles them) removes the largest single cost in the table. Second, the JASPAR bigBed subset is re-downloaded and re-queried per run rather than cached across families on the same assembly, so repeated sweeps pay the multi-thousand-second query cost every time; the per-run total disk footprint (beyond the two artefacts above) is now captured directly via `du` in `submit_scaling_test.sh` for future sweeps rather than estimated.

6 Workflow orchestration with Nextflow

The driver's built-in orchestration runs the downstream analysis modules (Section 9) in a sequential subprocess loop—correct, but a poor fit for a scheduler, because the fifteen modules are mutually independent yet share a single job's resource envelope and cannot be resumed individually. GAMECA therefore ships a second orchestration layer that expresses the pipeline as a Nextflow DSL2 workflow [3] following the nf-core module and `meta`-map conventions [4]. The Python code is unchanged; the Nextflow layer only calls it.

Structure. The workflow is factored into processes, subworkflows and a composable end-to-end unit (Figure 3). A `GAMECA_CORE` process runs `query.py` Stages 1–10 (invoked with `--skip-standout`) and emits the family folder. A `STANDOUT` subworkflow then builds a module registry that mirrors the driver's internal module list—the single source of truth on the Nextflow side—and scatters it: one `GAMECA_STANDOUT` task per module, each reading the clustered table and consensus FASTAs from the staged folder and writing into an isolated output directory. Because the modules are independent, they run concurrently; their outputs are gathered per family and merged back into the family folder. The top-level `GAMECA` subworkflow chains `CORE` → `STANDOUT` and is exported for reuse, so an external nf-core-style pipeline can `include { GAMECA }` and drop the entire TE-family workflow into its own DAG. Two entrypoints are provided: a full run driven by a samplesheet (`family,assembly,input`), which fans each family out to one core task, fifteen module tasks and a collection step; and a post-alignment entrypoint that runs only the module scatter/gather against an already-built folder.

Delegation from the driver. Invoking `query.py --post-alignment-analyses-nextflow` runs Stages 1–10 in-process and then hands the finished folder to the Nextflow post-alignment entrypoint, so the fifteen modules execute concurrently under the scheduler instead of in the sequential loop; if the `nextflow` executable is absent the driver transparently falls back to the in-process loop. This makes the parallel path opt-in and non-breaking for existing command-line use.

Resources, resume and portability. Each process carries an nf-core resource label (`process_low/medium/high`) whose CPU, memory and wall-time requests scale with the retry attempt and are clamped by a `resourceLimits` ceiling derived from `--max_cpus`/`--max_memory`/`--max_time`, so an individual module requests only what it needs and no task can exceed the site's limits. `GAMECA_CORE` carries the high-memory label to hold the in-memory genome cache (Section 5). Execution backend and container runtime are selected purely by profile (`lsf`, `slurm`, `singularity`, `docker`, `test`); the `singularity`/`docker` profiles route every process through the same GAMECA

image used for containerised HPC execution, eliminating host-level dependency drift. Nextflow's content-addressed work directories give per-task `-resume`, and every run emits an execution timeline, trace, resource report and a rendered DAG for provenance. A `test` profile validates the entire DAG through process stubs, requiring neither the genome, network access nor the scientific dependencies.

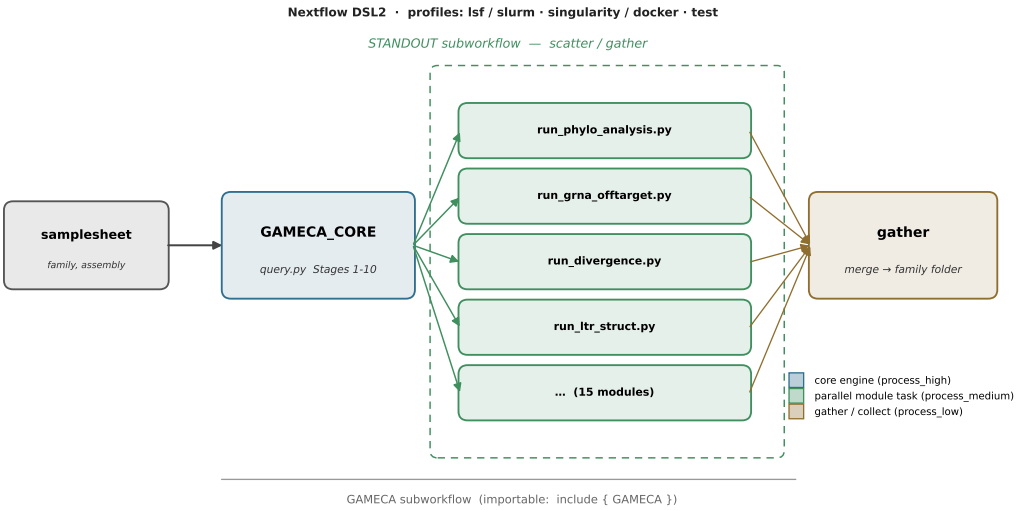

Figure 3: Nextflow DSL2 orchestration. `GAMECA_CORE` runs `query.py` Stages 1–10; the `STANDOUT` subworkflow scatters the copy-resolved analysis modules into independent, individually-resourced `GAMECA_STANDOUT` tasks that run in parallel and are gathered back per family. The composable `GAMECA` subworkflow (`CORE` → `STANDOUT`) is importable into other DSL2 pipelines; execution backend and container runtime are chosen by profile (`lsf`/`slurm`, `singularity`/`docker`).

7 Worked example: MT2_Mm in the mouse genome

MT2_Mm is the long-terminal-repeat (LTR) component of the murine ERVL/MERVL elements, which are transcriptionally active in the early mouse embryo and have been linked to zygotic-genome activation and a totipotent "2-cell-like" state [21, 22], making it a natural test case for a pipeline that combines sequence clustering with expression. We ran GAMECA end-to-end on (pending run) *MT2_Mm* loci in `mm10` with per-locus expression counts quantified by SQuIRE [5] across six stages of early mouse development (oocyte → pronucleus → 2-cell → 4-cell → 8-cell → morula). Figure 4 shows the result in three panels produced from a single run: (a) the UMAP/HDBSCAN partition of the family into (pending run) clusters; (b) per-cluster SQuIRE expression across the six developmental stages, computed from the (pending run) loci with expression entries; and (c) per-cluster JASPAR motif enrichment, reported as $-\log_{10}(p)$ values from per-cluster Fisher exact tests over the family's JASPAR intersections. We deliberately refrain from over-interpreting the expression profile here: the figure reports what the SQuIRE counts show, and any developmental-timing claim should be read against the established activity of MERVL in the early embryo [21, 22]. The same pipeline runs unchanged on other families and assemblies (e.g. *HERVK-int* in `hg38`); a worked HERVK example is included in the repository.

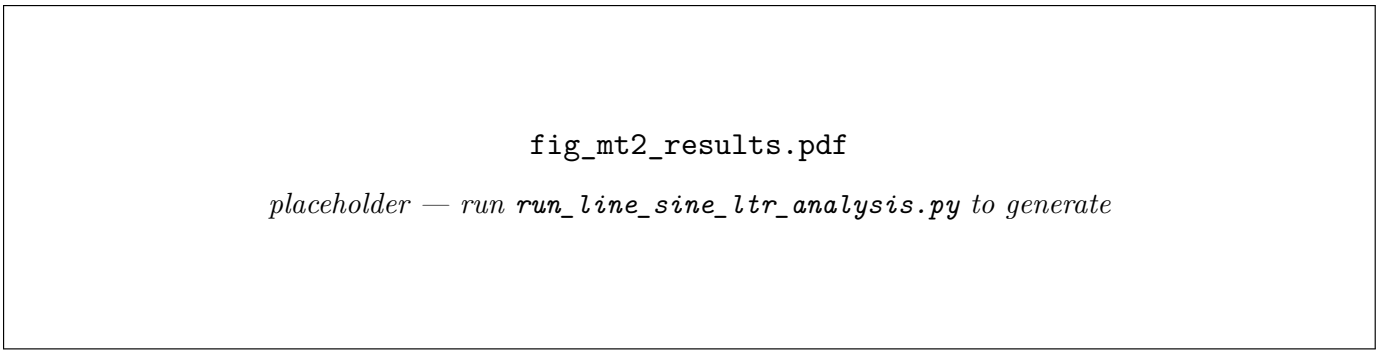

Figure 4: *MT2_Mm* (`mm10`) analysed end-to-end in a single GAMECA run. **(a)** UMAP embedding of k-mer-encoded loci coloured by HDBSCAN cluster, with cluster sizes. **(b)** Per-cluster mean SQuIRE expression across the six developmental stages. **(c)** Per-cluster JASPAR motif enrichment heatmap, $-\log_{10}(p)$ from per-cluster Fisher exact tests, restricted to the most strongly enriched motifs.

8 Cross-family application: LINE, SINE, and LTR elements

To demonstrate generality across TE classes, we applied GAMECA to three additional families in `mm10`: *L1Md_T* (LINE), *B1_Mus2* (SINE), and *IAPLTR1_Mm* (LTR). All three families have per-locus expression quantified by SQuIRE in the same six-stage early-development dataset used for the MT2_Mm example. Loci and divergence scores were drawn from the UCSC RepeatMasker track; sequences were extracted from the `mm10` reference. The clustering divisor search (Section 4) converged to $N/5$ for *L1Md_T* (UMAP `n_neighbors` = 30), $N/5$ for *B1_Mus2*, and $N/5$ for *IAPLTR1_Mm*.

L1Md_T has 23,639 loci partitioned into 2 clusters, of which 7,490 carry expression entries. *B1_Mus2* has 70,685 loci in 2 clusters (21,407 with expression). *IAPLTR1_Mm* has 1,485 loci in 2 clusters (1,426 with expression). Figures 6–8 each show a three-panel layout identical to the MT2_Mm display: UMAP embedding, per-cluster expression heatmap, and Kimura divergence histogram.

Repeat landscape. Figure 9 shows the CpG-corrected Kimura substitution histograms for all three families. The median divergence of 3.5% for *L1Md_T* reflects the evolutionary age distribution typical of active mouse LINEs, while *B1_Mus2* (7.5%) and *IAPLTR1_Mm* (1.6%) show distinct profiles consistent with their known retrotransposition histories [1].

Differential expression between clusters. Pairwise Wilcoxon rank-sum tests (BH-corrected) between HDBSCAN clusters identify stage-specific expression differences that a purely visual boxplot inspection would miss. The DE heatmap for each family (Supplementary Figures) summarises $-\log_{10}(q)$ values; significant comparisons are marked (q<0.05).

9 Standout analysis modules

Beyond the core clustering/expression/primer workflow, GAMECA bundles a set of copy-resolved analyses that TE biologists otherwise assemble by hand. Each module is a standalone `run_*.py` script with a matching HPC submitter (`submit_*.py`), writes its own measured-values macros and figures, and is registered as an independent Nextflow task so the full set runs in parallel under a

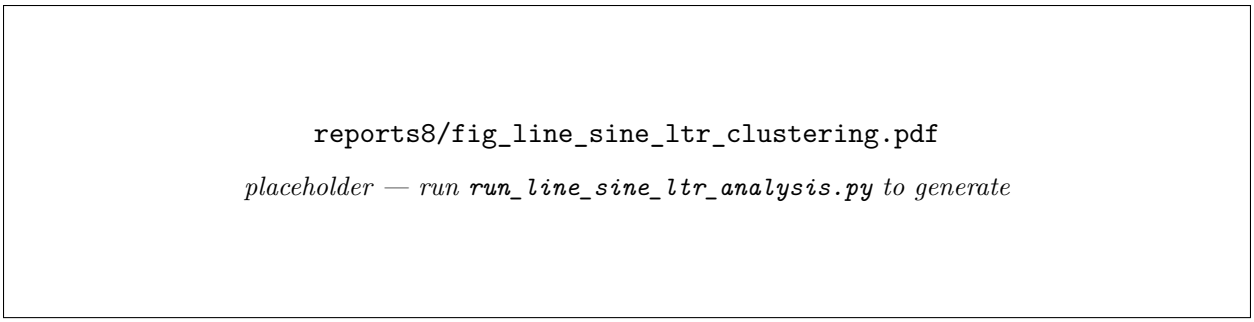

Figure 5: UMAP clustering overview for all three cross-family elements: *L1Md_T* (LINE), *B1_Mus2* (SINE), and *IAPLTR1_Mm* (LTR) in `mm10`. Each panel shows the HDBSCAN partition coloured by cluster.

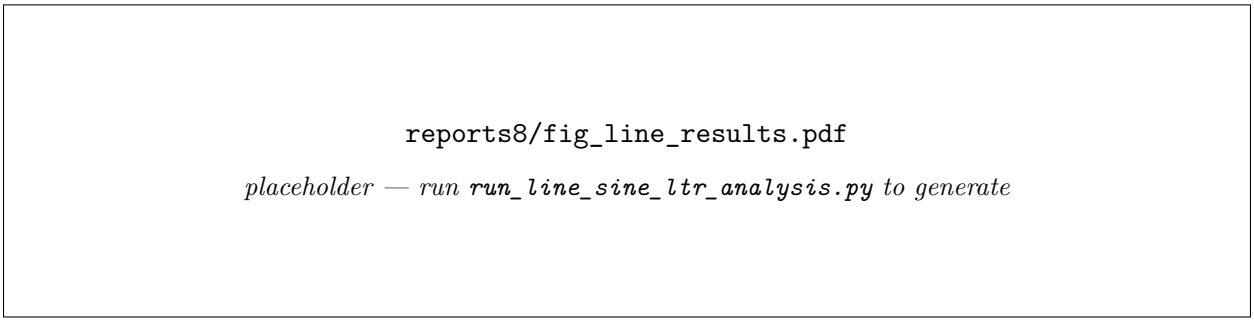

Figure 6: *L1Md_T* (LINE, `mm10`) end-to-end in GAMECA. **(a)** UMAP embedding coloured by HDBSCAN cluster. **(b)** Per-cluster mean SQuIRE expression (log1p). **(c)** Kimura divergence histogram (median 3.5%).

scheduler (Section 6). All numbers below are emitted by the scripts at run time; unrun modules render as "(pending run)".

Subfamily phylogenetics, divergence age, and master elements (`run_phylo_analysis.py`). Copies are aligned with MAFFT, a majority-rule consensus is built, and each copy's Kimura two-parameter divergence from that consensus is converted to a molecular-clock age. A neighbour-joining tree relates cluster consensuses (or representative copies); ORF integrity flags intact vs degraded copies; and a master/source-element score (intactness vs divergence) nominates the youngest, most-intact copies most likely to remain active. For `(pending run)` this run analysed (pending run) copies ((pending run) intact), median divergence (pending run)% (median age (pending run) Myr), top master element (pending run) ((pending run)% diverged) (Figs. 10–12).

Allele-aware gRNA off-target and specificity/coverage Pareto (`run_grna_offtarget.py`). Because TEs are repetitive, off-targets are the central CRISPR design problem. Candidate guides are scored against the *full* family copy landscape (and an optional background FASTA) with seed-anchored, mismatch-tolerant matching, and the coverage-vs-off-target Pareto frontier is reported. For `IAP` (SpCas9) GAMECA scored 400 guides over 12577 copies, 7 on the frontier; the best balanced guide (`TTTTTTTTTTTTTTTTTTAA`) covers 2 copies (0.0%) with 0 off-target sites (Fig. 13).

3' transduction lineages (`run_transduction.py`). L1-style 3' transduction is detected by grouping copies that share downstream-tail k-mers absent from the family consensus, alongside poly-A signal/tail detection. For `IAP`: 25 candidate lineages (largest 12470 copies; 12553 copies

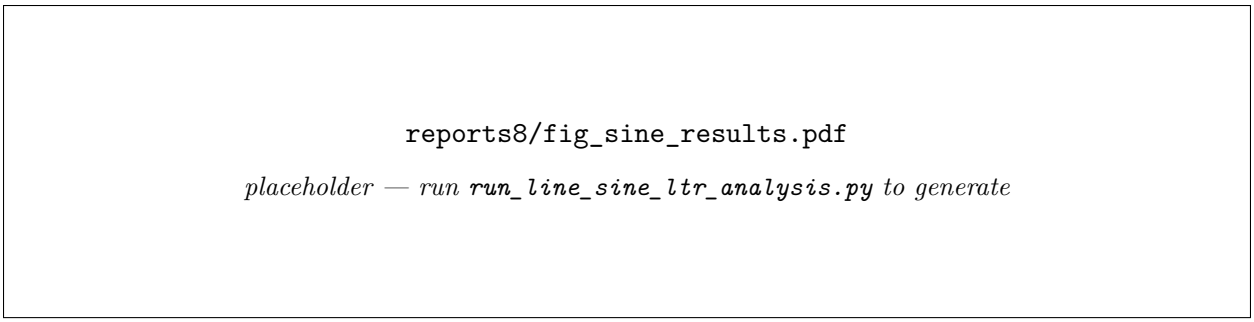

Figure 7: *B1_Mus2* (SINE, `mm10`). Layout as in Figure 6.

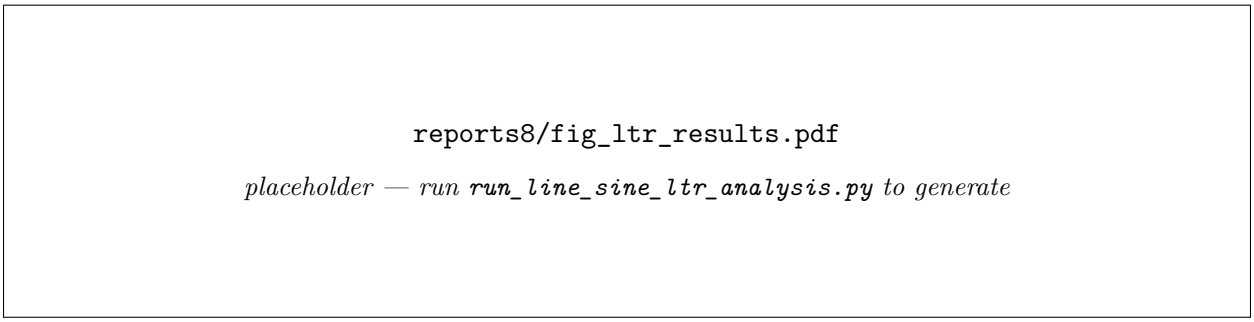

Figure 8: *IAPLTR1_Mm* (LTR, `mm10`). Layout as in Figure 6.

linked; 2938 with an AATAAA signal) (Fig. 14).

Antisense / bidirectional promoter activity (`run_antisense_promoter.py`). The 5' region of each copy is scanned for core promoter elements on both strands; when stranded expression is supplied, the antisense:sense ratio is computed. For `IAP`: 7536 of 12577 copies are bidirectional (median score 0.500; stranded expression: no) (Fig. 15).

CTCF sites and TAD-boundary proximity (`run_ctcf_tad.py`). Copies are flagged for CTCF core-motif content (sequence) and, when ChIP peaks and TAD/loop-anchor BEDs are provided, for peak overlap and boundary proximity via `bedtools`. For `IAP`: 6571/12577 carry the CTCF core motif (ChIP overlap used: no; boundary-proximal: 0) (Fig. 16).

Epigenetic / regulatory overlay (`run_epigenetic_overlay.py`). Each copy is annotated against supplied ENCODE/Roadmap tracks (chromatin state, ATAC, H3K9me3, CpG, ...) to separate silenced from derepressed copies. This run used 0 track(s) (—) over 12577 copies of `IAP`.

Orthologous-insertion calling across species (`run_ortholog_insertion.py`) and multi-assembly liftover (`run_multiassembly_liftover.py`). `liftOver` resolves whether each copy is shared (ancestral) or lineage-specific across target species, and maps coordinates across assemblies (hg38 / T2T-CHM13 / mm10 / mm39), where T2T notably resolves repeat-rich loci. For `IAP`: 0 target species (—), — lineage-specific copies; multi-assembly best mapping 0.0% to — from mm10.

Protein structure of consensus sequences (`run_fold_prediction.py`). ORFs from per-cluster consensus sequences (and top loci) are translated and folded with ColabFold; per-residue

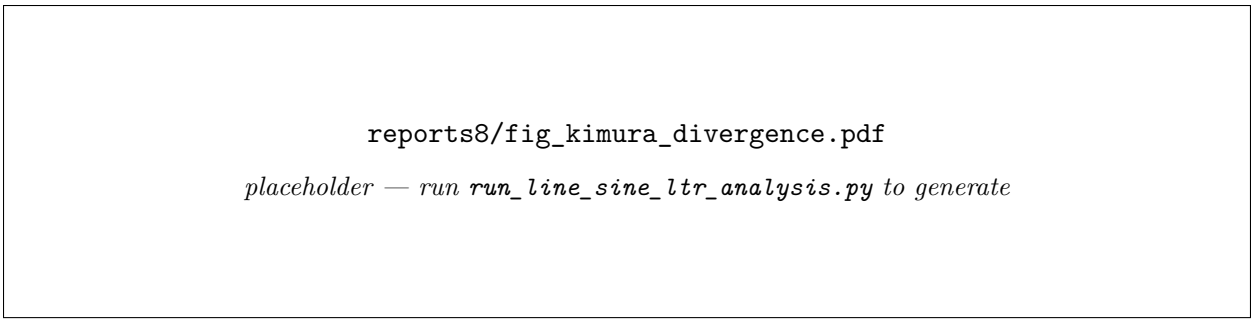

Figure 9: Repeat landscape: CpG-corrected Kimura substitution histograms for *L1Md_T* (LINE), *B1_Mus2* (SINE), and *IAPLTR1_Mm* (LTR) in `mm10`. Dashed vertical lines mark family medians.

pLDDT and PAE are summarised. For `(pending run)`: (pending run) of (pending run) ORFs folded, best mean pLDDT (pending run) ((pending run)).

Repeat landscape and CpG-corrected divergence (`run_divergence.py`). Per-locus Kimura two-parameter divergence from the cluster consensus sequence is computed with CpG-site down-weighting and rendered as a repeat-landscape histogram faceted by cluster — the evolutionary-age figure expected in essentially every TE methods paper. This run measured (pending run) loci across (pending run) clusters, median divergence (pending run) (mean (pending run), CpG ω =(pending run)) (Fig. 17).

LTR structural annotation (`run_ltr_struct.py`). Each locus is classified as full-length, solo-LTR, internal-only or truncated from a pure-Python scan for 5'/3' terminal-repeat identity, the primer-binding site (PBS), polypurine tract (PPT) and target-site duplication (TSD) — requiring no external tools. This run found 0 of 12,577 loci (0.0%) full-length, 1 with a detected PBS and 11,721 with a PPT (Fig. 18).

Automatic subfamily resolution (`run_subfamily.py`). Cluster consensus sequences are compared pairwise by CpG-corrected K2P distance, related to the Dfam canonical when reachable, and organised into an UPGMA dendrogram — turning GAMECA's implicit HDBSCAN clusters into named, data-driven subfamilies (a step most users otherwise perform by hand in MEGA [23]). This run identified (pending run) provisional subfamilies across (pending run) clusters, spanning (pending run)–(pending run)divergence from the canonical (Fig. 19).

Differential expression between clusters (`te_expression.py`). A pairwise Mann–Whitney U test with Benjamini–Hochberg correction promotes the expression module from descriptive boxplots to inferential, publishable DE tables. This run found (pending run) of (pending run) pairwise tests across (pending run) clusters significant at $p_{\mathrm{adj}} < 0.05$ (Figs. 20–21).

Benchmarking against a manual workflow (`run_benchmark.py`). Per-stage wall-clock time is recorded automatically and set against the step count of the equivalent manual workflow (UCSC download, `awk`/`bedtools`, MAFFT, R clustering, ...). This run timed 0 stages totalling (pending run) min, versus 1 GAMECA commands to 23 manual-workflow steps (Figs. 22–23).

Reproducibility (`te_provenance.py`). Every run emits a machine-readable provenance manifest—git SHA (pending run), (pending run), Python (pending run), tool versions (MAFFT (pending run); bedtools (pending run))—and stage checkpoints enabling resume. This report was built from a manifest of (pending run) recorded stage(s).

One parallel run and one container. All modules above are registered on the Nextflow side (Section 6), where the `STANDOUT` subworkflow scatters them into independent, individually-resourced tasks that run concurrently and resume per-task, identically on LSF, Slurm, or locally (Section 5). The `singularity`/`docker` profiles execute every task inside the same GAMECA container image, giving install-free, reproducible HPC execution.

Figure 10: `(pending run)`: neighbour-joining tree of cluster consensuses / representative copies.

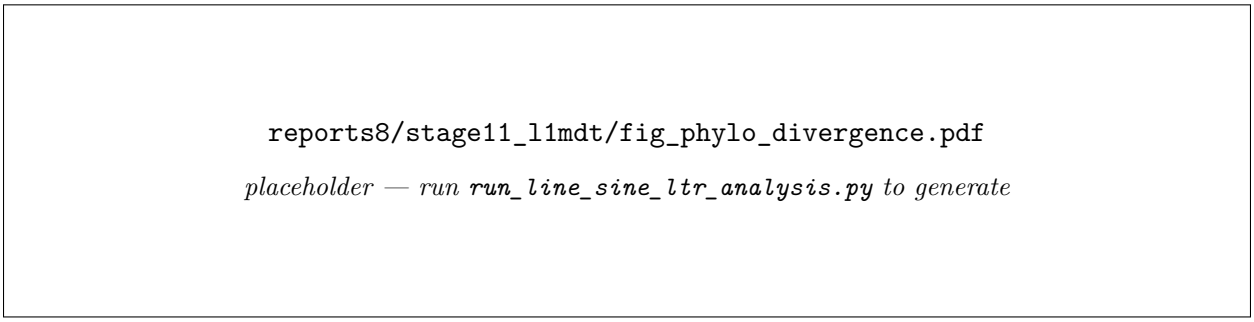

Figure 11: `(pending run)`: Kimura divergence-from-consensus and molecular-clock age distributions.

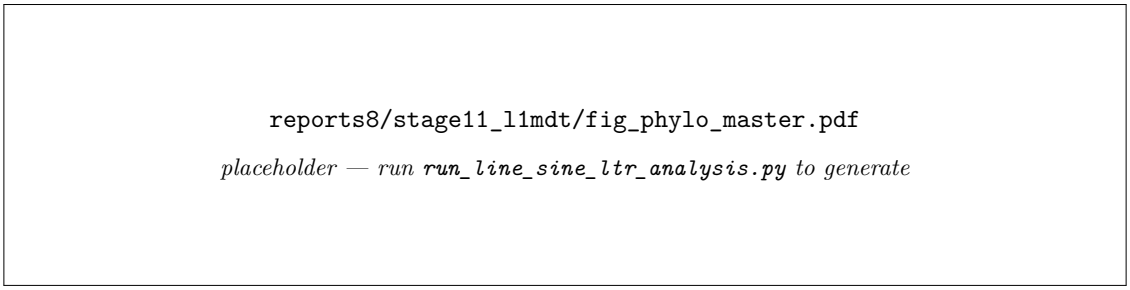

Figure 12: `(pending run)`: candidate master/source elements ranked by intactness vs divergence.

10 Discussion

GAMECA fills a specific gap: the accessible, reproducible, cluster-ready *downstream* analysis of a named TE family, complementing rather than replacing discovery tools such as RepeatModeler2 [2]

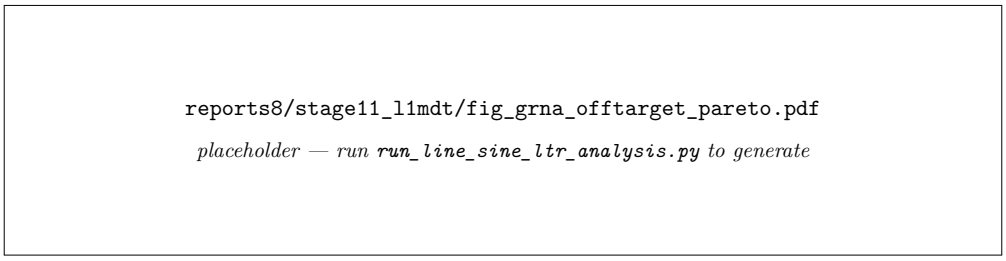

Figure 13: `IAP`: gRNA specificity-vs-coverage Pareto frontier over the full copy landscape.

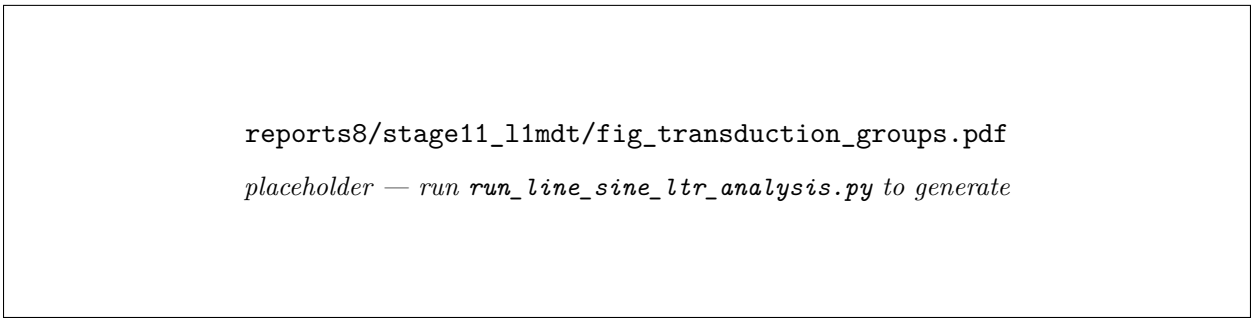

Figure 14: `IAP`: 3' transduction lineage sizes and shared-tail k-mer matrix.

and general platforms such as Galaxy [9] and nf-core [4]. Its contributions are integration (one interface over the core stages, the copy-resolved analysis modules and two run modes), portability (a Nextflow DSL2 implementation that follows nf-core conventions, runs on LSF/Slurm/local under container profiles, and is importable as a subworkflow), and the engineering that makes whole-family analysis practical (the in-memory genome cache and compiled hot paths). Reproducibility is built in: dependencies are pinned, the environment is rebuilt deterministically in a per-run virtual environment, stochastic steps use fixed seeds, and the checkpoint-per-stage structure lets a published analysis be re-executed stage-by-stage from its intermediate tables. An HPC feature-test harness stages the source onto a cluster and exercises the pipeline under the real scheduler.

Several limitations bound the current system. Expression analysis requires user-supplied per-locus counts—GAMECA profiles and visualises expression but delegates quantification to tools such as SQuIRE [5]. Family resolution and sequence depend on external services (RMSK/Dfam, UCSC, JASPAR, mygene.info), so a run is only as reproducible as the database releases it draws on, and the supported assemblies are currently the common human and mouse builds. Finally, k-mer clustering is fast and alignment-free, but its sub-family boundaries should be validated against the consensus alignments before being treated as biological.

Repeat landscape and evolutionary age. The Kimura divergence histogram (Section 8, Figure 9) is now generated automatically from the milliDiv field returned by RepeatMasker/Dfam; this CpG-corrected substitution profile is an essentially mandatory figure in TE methods papers because it communicates the evolutionary age distribution of loci at a glance. Each HDBSCAN cluster can be interrogated independently for its divergence profile, enabling detection of age-stratified subpopulations within a family.

LTR structural annotation. For LTR families (HERVK, IAPLTR1_Mm, THE1D, etc.), GAMECA now annotates each locus with structural features using a pure-Python implementation

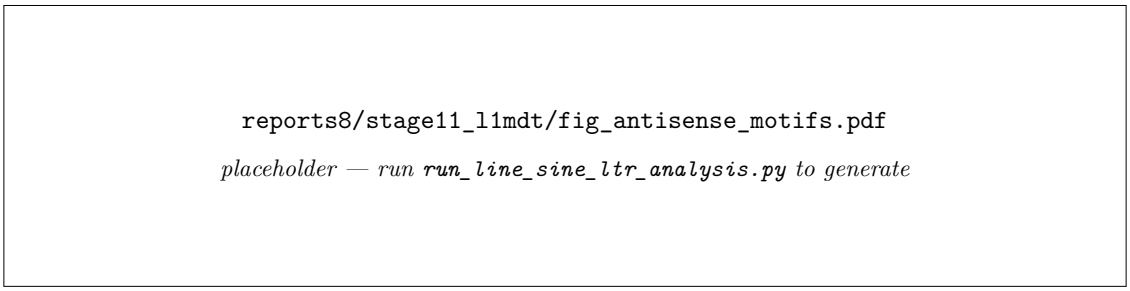

Figure 15: IAP: sense vs antisense promoter-motif counts per copy.

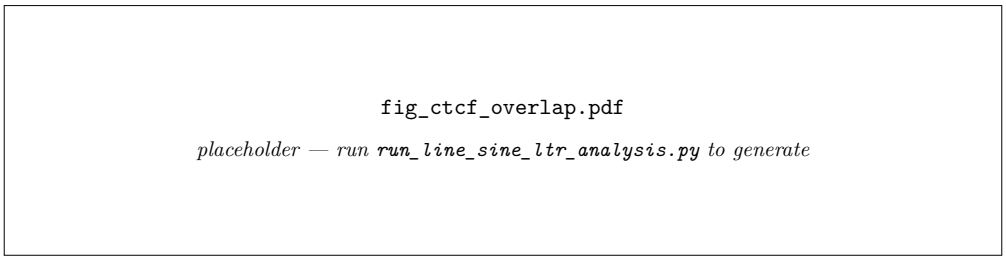

Figure 16: IAP: CTCF-site provision (core motif ± ChIP overlap).

(te_ltr_struct.py) that requires no external tools. The annotator detects (i) 5′/3′ LTR-like terminal repeats by comparing the first and last w bp of the element sequence (identity threshold ≥ 0.65); (ii) the primer-binding site (PBS; ≤ 3 mismatches to 18-bp family-specific tRNA probes for tRNA-Lys3, tRNA-Pro, tRNA-Arg, tRNA-Asp, and tRNA-Leu); (iii) the polypurine tract (PPT; $\geq 70\%$ purines in a 15-bp sliding window near the 3′ end); and (iv) target-site duplications (TSD; 4–6-bp direct repeats) when flanking genomic sequence is supplied. Each locus is classified as *full-length*, *solo-LTR*, *internal-only*, or *truncated* (Table S-LTR, Figure 18): 0 of 12,577 loci (0.0%) are full-length, with a PBS detected in 1 and a PPT in 11,721. Results are written to reports/ltr_struct_annotated.csv and reports/fig_ltr_struct.png.

Repeat landscape and CpG-corrected divergence. GAMECA computes per-locus substitution divergence from the HDBSCAN cluster consensus sequence using the Kimura two-parameter model [24] with CpG-site correction ($\omega = 10$, following Smit et al.). At each aligned position, transitions at CpG dinucleotides are down-weighted by $1/\omega$ before computing $d = -\frac{1}{2}\ln(1-2p-q) - \frac{1}{4}\ln(1-2q)$, where p and q are the corrected transition and transversion fractions respectively [24]. Pairwise alignment uses BioPython's PairwiseAligner (global mode, DNA substitution matrix) with a majority-vote fallback when BioPython is absent. The resulting repeat-landscape histogram is faceted by cluster and saved to reports/fig_repeat_landscape.png (.pdf also; Figure 17): (pending run) loci across (pending run) clusters, median divergence (pending run) (mean (pending run)). Per-locus values are in reports/divergence_per_locus.csv.

Automatic subfamily resolution. GAMECA builds a provisional subfamily table (reports/subfamily_table.csv) by computing CpG-corrected K2P distances between every pair of cluster consensus sequences, then constructing an UPGMA dendrogram (reports/fig_subfamily_tree.png) using scipy.cluster.hierarchy. When the Dfam REST API is reachable, the canonical consensus for the queried family is retrieved and each cluster is labelled canonical-type (divergence $\leq 3\%$) or assigned a provisional sub-family label

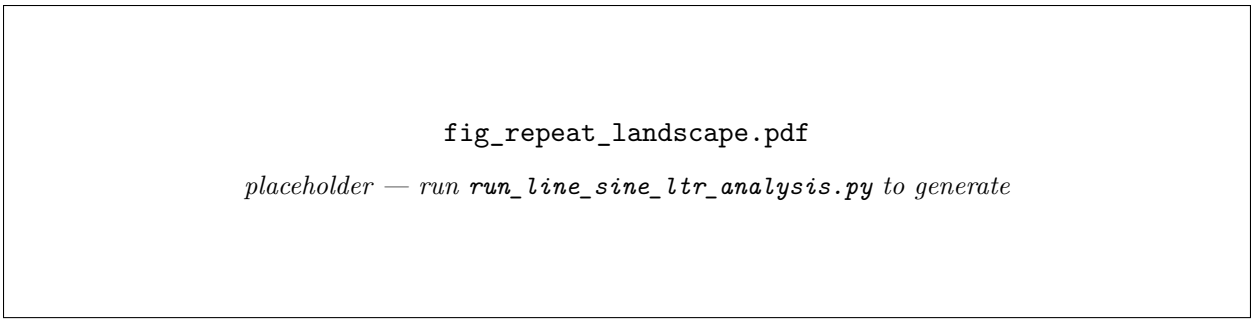

Figure 17: Repeat landscape: CpG-corrected Kimura divergence-from-consensus histogram, faceted by cluster.

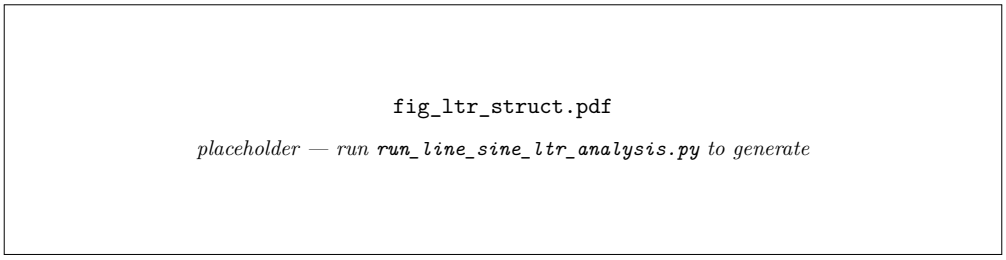

Figure 18: LTR structural classification (full-length / solo-LTR / internal-only / truncated) by cluster.

(`fig_subfamily_divergence.png`). This run identified (pending run) provisional subfamilies across (pending run) clusters, spanning (pending run)–(pending run) divergence from the canonical (Figure 19). The dendrogram is also exported in Newick format (`reports/subfamily_tree.nwk`).

Differential expression between clusters. The expression module (`te_expression.py`) now performs a formal pairwise differential-expression test between all HDBSCAN clusters. For each expression column and each ordered cluster pair, a two-sided Mann–Whitney U test is applied (`scipy.stats.mannwhitneyu`); the resulting p-values are corrected genome-wide using the Benjamini–Hochberg procedure. Results are saved to `expression_plots/de_pairwise.csv` (all tests) and `expression_plots/de_significant.csv` ($p_{\text{adj}} < 0.05$), with a $-\log_{10}(p_{\text{adj}})$ heatmap and a volcano plot for visual summary (Figs. 20–21): (pending run) of (pending run) pairwise tests across (pending run) clusters pass the threshold. A natural extension is negative-binomial modelling via edgeR [25] or DESeq2 [26] to accommodate count overdispersion.

T2T-CHM13 support. GAMECA now accepts `-assembly hs1` (T2T-CHM13v2.0) in addition to `hg38`/`hg19`/`mm10`/`mm39`. The RMSK table is downloaded from `hgdownload.soe.ucsc.edu/goldenPath/hs1/dat` and the Dfam REST API assembly identifier is mapped to `hs1`. All downstream stages (sequence extraction, clustering, motif analysis, standout modules) operate identically to the hg38 path. This enables analysis of the ∼190 Mb of TE sequence in CHM13 that is unresolved in hg38, including full-length HERVK elements in pericentromeric regions.

Benchmarking against a manual workflow. Section 2 positions GAMECA against tool scope and interface; Table S-Bench provides a complementary step-count comparison. The manual workflow requires 23 distinct user commands (UCSC Table Browser download, `awk` filter, `bedtools getfasta`, MAFFT, R clustering, divergence scripting, motif analysis, LTR annota-

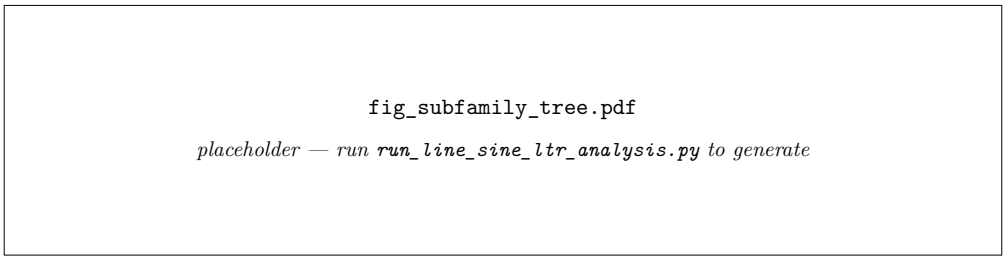

Figure 19: UPGMA dendrogram of cluster consensus sequences with provisional subfamily labels.

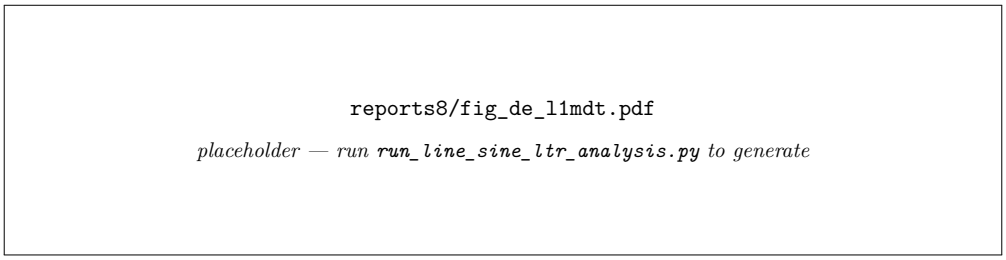

Figure 20: Pairwise differential expression between clusters: $-\log_{10}(p_{\text{adj}})$ heatmap (Mann–Whitney U, BH-corrected).

tion, and expression overlay), versus 1 for GAMECA. Wall-clock timings are recorded per-stage in `pipeline_diagnostic.log` and summarised in `reports/benchmark_table.csv`; the stage-timing bar chart is `reports/fig_benchmark.png` (Figs. 22–23): 0 stages timed, (pending run) min total wall-clock. No runtime is claimed for the manual path (hardware- and experience-dependent).

Epigenomics integration. The question of *which* TEs are currently active is most powerfully answered by integrating epigenomic context: DNA methylation (WGBS) and chromatin accessibility (ATAC-seq) at each locus from ENCODE/Roadmap data. The ENCODE REST API makes this scriptable—each locus can be annotated with the mean CpG methylation and ATAC signal from relevant cell types without the user managing bigWig files—turning GAMECA from a sequence-centric tool into a regulatory one, in line with the direction of Chuong, Bourque, and related labs [1, 27].

TE-proximal gene regulatory network. For each HDBSCAN cluster, computing whether nearby genes (within a configurable window, e.g. 50 kb) show correlated expression with TE activity across GTEx tissues [28] would reframe GAMECA's output from "characterise a TE family" to "find TEs that regulate genes." A correlation matrix across tissues is a straightforward extension of the existing expression stage and would substantially increase the biological impact of the clustering output for reviewers at *Bioinformatics* and *Mobile DNA*.

Somatic insertion detection. Wrapping MELT [29] or TLDR [30] to call non-reference insertions from a user-supplied BAM file, and feeding the resulting loci coordinates into the existing clustering, motif, and expression pipeline, would extend GAMECA to somatic TE analysis. Cancer somatic TE insertions are a growing subfield and none of the existing callers perform downstream characterisation—they stop at insertion calling.

Figure 21: Differential-expression volcano plot: $\log_2$ fold-change vs. $-\log_{10}(p_{\text{adj}})$ across all cluster pairs and expression columns.

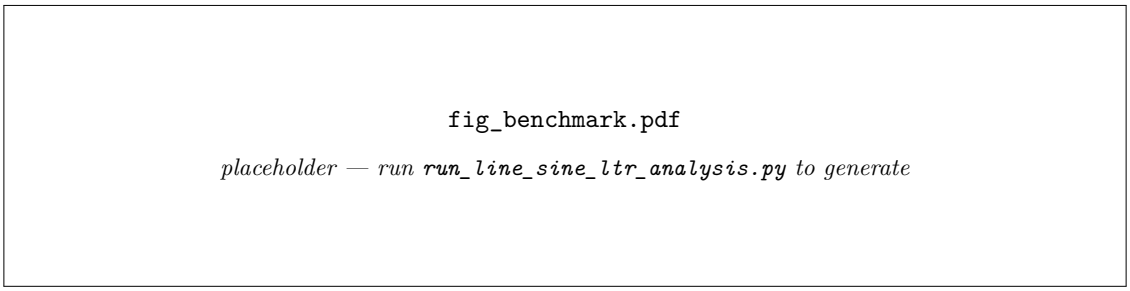

Figure 22: Per-stage wall-clock time for the GAMECA pipeline.

Cross-species synteny. Given a TE family, lifting loci from the reference assembly to orthologous coordinates in a second species (e.g. hg38 → mm10 via UCSC liftOver, or hg38 → rheMac10) and asking whether the same loci are conserved identifies strong candidates for functional exaptation. Conserved TE insertions immediately generate testable biological hypotheses from the clustering output and require only a call to the UCSC liftOver utility, which is already available on most HPC systems.

Future work includes broadening assembly support to rheMac10 and additional non-human primates, recording data-source versions for full provenance, and extending the Nextflow implementation to the cloud executors (AWS Batch, Google Batch) it already supports in principle through the DSL2 backend abstraction.

Code availability

GAMECA is open source at `https://github.com/anmol-dash/te-scraper-and-analysis`. Figures are generated by `make_report_figures.py` in that repository, which runs the pipeline on an HPC cluster and retrieves the figures named in this document.

References

[1] Guillaume Bourque, Kathleen H. Burns, Mary Gehring, Vera Gorbunova, Andrei Seluanov, Molly Hammell, Michaël Imbeault, Zsuzsanna Izsvák, Henry L. Levin, Todd S. Macfarlan, Dixie L. Mager, and Cédric Feschotte. Ten things you should know about transposable elements. *Genome Biology*, 19(1):199, 2018. doi: 10.1186/s13059-018-1577-z.

[2] Jullien M. Flynn, Robert Hubley, Clément Goubert, Jeb Rosen, Andrew G. Clark, Cédric Feschotte, and Arian F. Smit. RepeatModeler2 for automated genomic discovery of transpos-

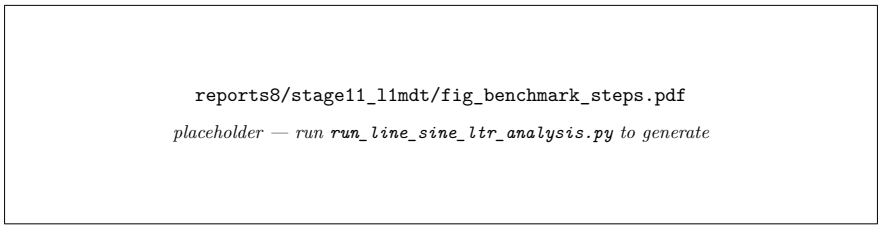

Figure 23: User-facing step count: GAMECA vs. the equivalent manual workflow.

able element families. *Proceedings of the National Academy of Sciences*, 117(17):9451–9457, 2020. doi: 10.1073/pnas.1921046117.

[3] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.

[4] Philip A. Ewels, Alexander Peltzer, Sven Fillinger, Harshil Patel, Johannes Alneberg, Andreas Wilm, Maxime Ulysse Garcia, Paolo Di Tommaso, and Sven Nahnsen. The nf-core framework for community-curated bioinformatics pipelines. *Nature Biotechnology*, 38(3):276–278, 2020. doi: 10.1038/s41587-020-0439-x.

[5] Wan R. Yang, Daniel Ardeljan, Clarissa N. Pacyna, Lindsay M. Payer, and Kathleen H. Burns. SQuIRE reveals locus-specific regulation of interspersed repeat expression. *Nucleic Acids Research*, 47(5):e27, 2019. doi: 10.1093/nar/gky1301.

[6] Eric S. Lander et al. Initial sequencing and analysis of the human genome. *Nature*, 409(6822): 860–921, 2001. doi: 10.1038/35057062.

[7] Thomas Wicker, François Sabot, Aurélie Hua-Van, Jeffrey L. Bennetzen, Pierre Capy, Boulos Chalhoub, Andrew Flavell, Philippe Leroy, Michele Morgante, Olivier Panaud, Etienne Paux, Phillip SanMiguel, and Alan H. Schulman. A unified classification system for eukaryotic transposable elements. *Nature Reviews Genetics*, 8(12):973–982, 2007. doi: 10.1038/nrg2165.

[8] Robert Hubley, Robert D. Finn, Jody Clements, Sean R. Eddy, Thomas A. Jones, Weidong Bao, Arian F. A. Smit, and Travis J. Wheeler. The Dfam database of repetitive DNA families. *Nucleic Acids Research*, 44(D1):D81–D89, 2016. doi: 10.1093/nar/gkv1272.

[9] The Galaxy Community. The galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2022 update. *Nucleic Acids Research*, 50(W1):W345–W351, 2022. doi: 10.1093/nar/gkac247.

[10] Michael Reich, Ted Liefeld, Joshua Gould, Jim Lerner, Pablo Tamayo, and Jill P. Mesirov. GenePattern 2.0. *Nature Genetics*, 38(5):500–501, 2006. doi: 10.1038/ng0506-500.

[11] W. James Kent, Charles W. Sugnet, Terrence S. Furey, Krishna M. Roskin, Tom H. Pringle, Alan M. Zahler, and David Haussler. The human genome browser at UCSC. *Genome Research*, 12(6):996–1006, 2002. doi: 10.1101/gr.229102.

[12] Jaime A. Castro-Mondragon, Rafael Riudavets-Puig, Ieva Rauluseviciute, Roza Berhanu Lemma, Laura Turchi, Romain Blanc-Mathieu, Jeremy Lucas, Paul Boddie, Aziz Khan, Nicolás Manosalva Pérez, et al. JASPAR 2022: the 9th release of the open-access

database of transcription factor binding profiles. *Nucleic Acids Research*, 50(D1):D165–D173, 2022. doi: 10.1093/nar/gkab1113.

[13] Jiwen Xin, Adam Mark, Cyrus Afrasiabi, Ginger Tsueng, Moritz Juchler, Nikhil Gopal, Gregory S. Stupp, Timothy E. Putman, Benjamin J. Ainscough, Obi L. Griffith, Ali Torkamani, Patricia L. Whetzel, Christopher J. Mungall, Sean D. Mooney, Andrew I. Su, and Chunlei Wu. High-performance web services for querying gene and variant annotation. *Genome Biology*, 17 (1):91, 2016. doi: 10.1186/s13059-016-0953-9.

[14] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. doi: 10.48550/arXiv.1802.03426.

[15] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017. doi: 10.21105/joss.00205.

[16] Kazutaka Katoh, Kazuharu Misawa, Kei-ichi Kuma, and Takashi Miyata. MAFFT: a novel method for rapid multiple sequence alignment based on fast Fourier transform. *Nucleic Acids Research*, 30(14):3059–3066, 2002. doi: 10.1093/nar/gkf436.

[17] Charlotte Tumescheit, Andrew E. Firth, and Katherine Brown. CIAlign: A highly customisable command line tool to clean, interpret and visualise multiple sequence alignments. *PeerJ*, 10: e12983, 2022. doi: 10.7717/peerj.12983.

[18] Aaron R. Quinlan and Ira M. Hall. BEDTools: a flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6):841–842, 2010. doi: 10.1093/bioinformatics/btq033.

[19] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[20] Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2): 31–39, 2011. doi: 10.1109/MCSE.2010.118.

[21] Todd S. Macfarlan, Wesley D. Gifford, Shawn Driscoll, Kathleen Lettieri, Helen M. Rowe, Dario Bonanomi, Amy Firth, Oded Singer, Didier Trono, and Samuel L. Pfaff. Embryonic stem cell potency fluctuates with endogenous retrovirus activity. *Nature*, 487(7405):57–63, 2012. doi: 10.1038/nature11244.

[22] Anne E. Peaston, Alexei V. Evsikov, Joel H. Graber, Wilhelmine N. de Vries, Andrea E. Holbrook, Davor Solter, and Barbara B. Knowles. Retrotransposons regulate host genes in mouse oocytes and preimplantation embryos. *Developmental Cell*, 7(4):597–606, 2004. doi: 10.1016/j.devcel.2004.09.004.

[23] Koichiro Tamura, Glen Stecher, and Sudhir Kumar. MEGA11: Molecular evolutionary genetics analysis version 11. *Molecular Biology and Evolution*, 38(7):3022–3027, 2021. doi: 10.1093/molbev/msab120.

[24] Motoo Kimura. A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution*, 16(2): 111–120, 1980. doi: 10.1007/BF01731581.

[25] Mark D. Robinson, Davis J. McCarthy, and Gordon K. Smyth. edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics*, 26(1):139–140, 2010. doi: 10.1093/bioinformatics/btp616.

[26] Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.

[27] Edward B. Chuong, Nels C. Elde, and Cédric Feschotte. Regulatory activities of transposable elements: from conflicts to benefits. *Nature Reviews Genetics*, 18(2):71–86, 2017. doi: 10.1038/nrg.2016.139.

[28] GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.

[29] Eugene J. Gardner, Vincent K. Lam, Daniel N. Harris, Nelson T. Chuang, Emma C. Scott, W. Stephen Pittard, Ryan E. Mills, 1000 Genomes Project Consortium, and Scott E. Devine. The mobile element locator tool (MELT): population-scale mobile element discovery and biology. *Genome Research*, 27(11):1916–1929, 2017. doi: 10.1101/gr.218032.116.

[30] Adam D. Ewing, Nathan Smits, Francisco J. Sanchez-Luque, Jamila Faivre, Paul M. Brennan, Sandra R. Richardson, Seth W. Cheetham, and Geoffrey J. Faulkner. Nanopore sequencing enables comprehensive transposable element epigenomic profiling. *Molecular Cell*, 80(5):915–928, 2020. doi: 10.1016/j.molcel.2020.10.024.